Final Project method 2

Jack

2026-04-16

Method 2

Model 1

```
library(coda)
```

```
## Warning: package 'coda' was built under R version 4.3.3
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(tidyr)
library(cmdstanr)
```

```
## This is cmdstanr version 0.9.0
```

```
## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr
```

```
## - CmdStan path: /Users/jackyip/.cmdstan/cmdstan-2.38.0
```

```
## - CmdStan version: 2.38.0
```

```
library(loo)
```

```
## Warning: package 'loo' was built under R version 4.3.3
```

```
## This is loo version 2.8.0
```

```
## - Online documentation and vignettes at mc-stan.org/loo
```

```
## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use
the 'cores' argument or set options(mc.cores = NUM_CORES) for an entire session.
```

```
library(posterior)
```

```
## This is posterior version 1.7.0
```

```
##
## Attaching package: 'posterior'
```

```
## The following object is masked from 'package:loo':
##
##     gpdfit
```

```
## The following objects are masked from 'package:stats':
##
##     mad, sd, var
```

```
## The following objects are masked from 'package:base':
##
##     %in%, match
```

```
housing_data <- read.csv("~/Downloads/melb_data.csv") %>%
  mutate(
    distance_z = as.numeric(scale(Distance)),
    buildingarea_z = as.numeric(scale(BuildingArea)),
    log_price = log(Price)
  ) %>%
  filter(
    !is.na(log_price),
    !is.na(distance_z),
    !is.na(buildingarea_z)
  )

y_m1 <- housing_data$log_price
x_dist_m1 <- housing_data$distance_z
x_area_m1 <- housing_data$buildingarea_z

# Log posterior for Model 1
log_posterior_model1 <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]

  sigma <- exp(eta)

  if (!is.finite(sigma) || sigma <= 0) return(-Inf)

  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  if (any(!is.finite(mu))) return(-Inf)

  log_prior <-
    dnorm(beta0, mean = 0, sd = 10, log = TRUE) +
    dnorm(beta1, mean = 0, sd = 1, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) +
    eta

  log_lik <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_lik
}

#  One MH step
mh_step_model1 <- function(current, current_lp, proposal_sds, y, distance, buildingarea)
{
  proposal <- current + rnorm(length(current), mean = 0, sd = proposal_sds)

  proposal_lp <- log_posterior_model1(
    theta = proposal,
    y = y,
    distance = distance,
```

```
    buildingarea = buildingarea
  )

  log_alpha <- proposal_lp - current_lp
  accepted <- is.finite(proposal_lp) && (log(runif(1)) < log_alpha)

  if (accepted) {
    list(state = proposal, lp = proposal_lp, accepted = TRUE)
  } else {
    list(state = current, lp = current_lp, accepted = FALSE)
  }
}

# Mixture MH sampler
mh_mixture_model1 <- function(y, distance, buildingarea, init,
                              n_iter = 20000,
                              proposal_sds_small = c(0.02, 0.005, 0.005, 0.01),
                              proposal_sds_large = c(0.08, 0.02, 0.02, 0.04),
                              mix_prob_small = 0.7) {
  samples <- matrix(NA_real_, nrow = n_iter, ncol = 4)
  colnames(samples) <- c("beta0", "beta1", "beta2", "eta")

  current <- init
  current_lp <- log_posterior_model1(current, y, distance, buildingarea)

  accepted <- logical(n_iter)

  for (i in seq_len(n_iter)) {
    use_small <- runif(1) < mix_prob_small
    sds <- if (use_small) proposal_sds_small else proposal_sds_large

    out <- mh_step_model1(
      current = current,
      current_lp = current_lp,
      proposal_sds = sds,
      y = y,
      distance = distance,
      buildingarea = buildingarea
    )

    current <- out$state
    current_lp <- out$lp
    accepted[i] <- out$accepted
    samples[i, ] <- current
  }

  list(
    samples = samples,
    acceptance_rate = mean(accepted)
  )
}
```

```r
# Run 4 MH chains
set.seed(123)

n_chains <- 4
n_iter_mh <- 20000
burn_in <- 5000
thin <- 5

init_list <- list(
  c(13.5, -0.05,  0.05, log(0.6)),
  c(14.0, -0.15,  0.10, log(0.5)),
  c(13.8, -0.10,  0.00, log(0.7)),
  c(13.6, -0.20,  0.08, log(0.4))
)

mh_fits <- vector("list", n_chains)

for (ch in 1:n_chains) {
  mh_fits[[ch]] <- mh_mixture_model1(
    y = y_m1,
    distance = x_dist_m1,
    buildingarea = x_area_m1,
    init = init_list[[ch]],
    n_iter = n_iter_mh
  )
}

sapply(mh_fits, function(x) x$acceptance_rate)
```

```
## [1] 0.1543 0.1508 0.1477 0.1480
```

```r
mean(sapply(mh_fits, function(x) x$acceptance_rate))
```

```
## [1] 0.1502
```

```r
#post-burn-in draws
mh_draws_list <- lapply(mh_fits, function(fit) {
  out <- as.data.frame(fit$samples)
  out <- out[(burn_in + 1):nrow(out), ]
  out <- out[seq(1, nrow(out), by = thin), ]
  out$sigma <- exp(out$eta)
  out
})

mh_draws <- bind_rows(mh_draws_list, .id = "chain")
mh_draws$chain <- as.integer(mh_draws$chain)

# MH summaries
mh_summary <- mh_draws %>%
  summarise(
    beta0_mean = mean(beta0),
    beta1_mean = mean(beta1),
    beta2_mean = mean(beta2),
    sigma_mean = mean(sigma),
    beta0_q5 = quantile(beta0, 0.05),
    beta1_q5 = quantile(beta1, 0.05),
    beta2_q5 = quantile(beta2, 0.05),
    sigma_q5 = quantile(sigma, 0.05),
    beta0_q95 = quantile(beta0, 0.95),
    beta1_q95 = quantile(beta1, 0.95),
    beta2_q95 = quantile(beta2, 0.95),
    sigma_q95 = quantile(sigma, 0.95)
  )

mh_summary
```

```
##   beta0_mean  beta1_mean beta2_mean sigma_mean beta0_q5   beta1_q5   beta2_q5
## 1   13.73927 -0.09273711 0.06142306  0.5309578  13.7288 -0.1032476 0.05157917
##    sigma_q5 beta0_q95   beta1_q95  beta2_q95 sigma_q95
## 1 0.5236556  13.74987 -0.08282485 0.07166787  0.538661
```

```
mh_table <- data.frame(
  variable = c("beta0", "beta1", "beta2", "sigma"),
  mean = c(mean(mh_draws$beta0), mean(mh_draws$beta1), mean(mh_draws$beta2), mean(mh_dra
ws$sigma)),
  median = c(median(mh_draws$beta0), median(mh_draws$beta1), median(mh_draws$beta2), med
ian(mh_draws$sigma)),
  sd = c(sd(mh_draws$beta0), sd(mh_draws$beta1), sd(mh_draws$beta2), sd(mh_draws$sigm
a)),
  q5 = c(quantile(mh_draws$beta0, 0.05), quantile(mh_draws$beta1, 0.05), quantile(mh_dra
ws$beta2, 0.05), quantile(mh_draws$sigma, 0.05)),
  q95 = c(quantile(mh_draws$beta0, 0.95), quantile(mh_draws$beta1, 0.95), quantile(mh_dr
aws$beta2, 0.95), quantile(mh_draws$sigma, 0.95))
)

mh_table
```

```
##   variable        mean      median          sd          q5         q95
## 1    beta0 13.73926741 13.73924289 0.006304899 13.72879678 13.74987281
## 2    beta1 -0.09273711 -0.09266675 0.006220925 -0.10324756 -0.08282485
## 3    beta2  0.06142306  0.06146047 0.006063590  0.05157917  0.07166787
## 4    sigma  0.53095776  0.53089151 0.004508547  0.52365559  0.53866101
```

```
# Convergence checks for MH
mh_coda <- coda::mcmc.list(lapply(mh_draws_list, function(df) {
  coda::mcmc(df[, c("beta0", "beta1", "beta2", "sigma")])
}))

coda::gelman.diag(mh_coda, autoburnin = FALSE)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## beta0          1       1.00
## beta1          1       1.00
## beta2          1       1.00
## sigma          1       1.01
##
## Multivariate psrf
##
## 1
```

```
coda::effectiveSize(mh_coda)
```

```
##    beta0    beta1    beta2    sigma
## 4827.010 1238.832 1312.499 2127.762
```

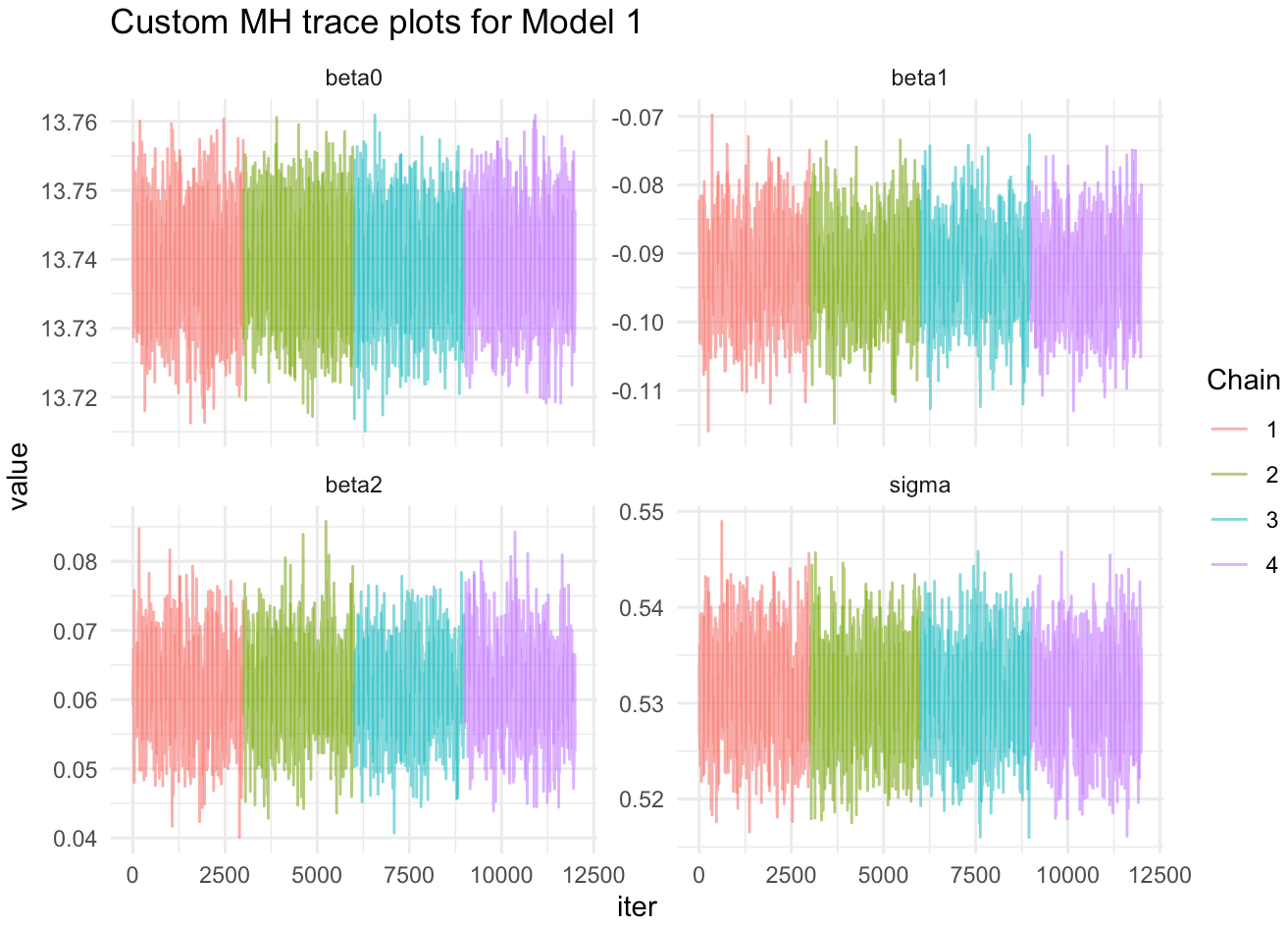
```
gelman.diag(mh_coda, autoburnin = FALSE)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## beta0          1       1.00
## beta1          1       1.00
## beta2          1       1.00
## sigma          1       1.01
##
## Multivariate psrf
##
## 1
```

```
effectiveSize(mh_coda)
```

```
##    beta0    beta1    beta2    sigma
## 4827.010 1238.832 1312.499 2127.762
```

```
# Trace plots for MH
mh_long <- mh_draws %>%
  select(chain, beta0, beta1, beta2, sigma) %>%
  mutate(iter = row_number()) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value")

ggplot(mh_long, aes(x = iter, y = value, colour = factor(chain))) +
  geom_line(alpha = 0.6) +
  facet_wrap(~ parameter, scales = "free_y", ncol = 2) +
  labs(title = "Custom MH trace plots for Model 1", colour = "Chain") +
  theme_minimal()
```


Custom MH trace plots for Model 1
beta0
beta1
beta2
sigma
value
iter
Chain
1
2
3
4
13.76
13.75
13.74
13.73
13.72
-0.07
-0.08
-0.09
-0.10
-0.11
0.08
0.07
0.06
0.05
0.04
0.55
0.54
0.53
0.52
0
2500
5000
7500
10000
12500

```
mod1 <- cmdstan_model("~/Model1.stan")
mod2 <- cmdstan_model("~/Part2_Model2_Hierarchical.stan")

part1_data <- housing_data

stan_data_model1 <- list(
  N = nrow(part1_data),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z
)

part1_data$region_id <- as.integer(as.factor(part1_data$Regionname))

stan_data_model2 <- list(
  N = nrow(part1_data),
  R = length(unique(part1_data$region_id)),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z,
  region = part1_data$region_id
)

fit_m1 <- mod1$sample(
  data = stan_data_model1,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.95,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 10.8 seconds.
## Chain 3 finished in 11.0 seconds.
## Chain 2 finished in 12.6 seconds.
## Chain 4 finished in 13.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 12.0 seconds.
## Total execution time: 13.8 seconds.
```

```
fit_m2 <- mod2$sample(
  data = stan_data_model2,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.99,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 2 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 2 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 2
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 3 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 3
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

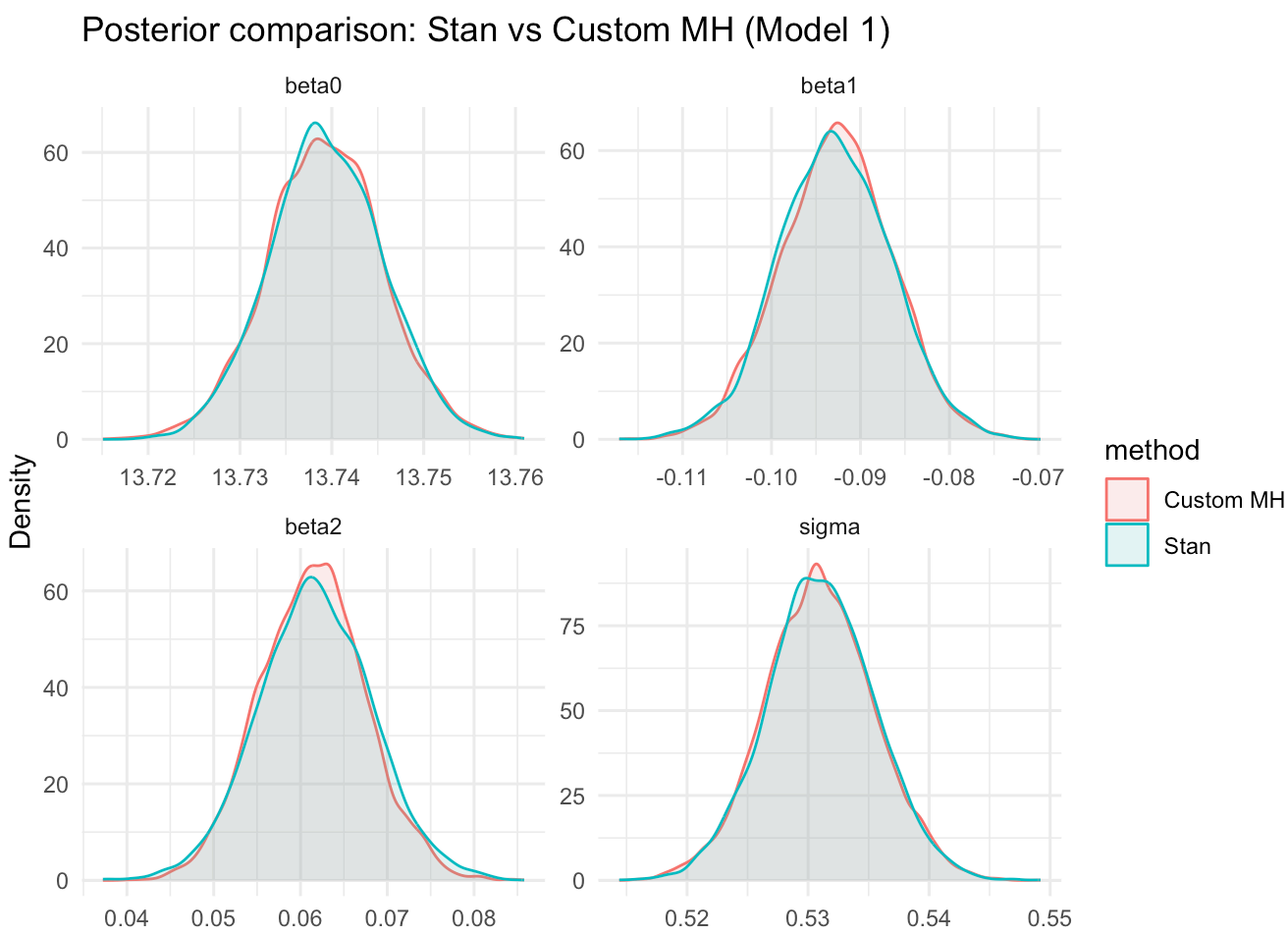
```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 3 finished in 170.6 seconds.
## Chain 1 finished in 232.9 seconds.
## Chain 4 finished in 240.0 seconds.
## Chain 2 finished in 366.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 252.4 seconds.
## Total execution time: 366.1 seconds.
```

```
# Compare MH vs Stan for Model 1
stan_m1_draws <- fit_m1$draws(c("beta0", "beta1", "beta2", "sigma"), format = "df") %>%
  as.data.frame() %>%
  select(beta0, beta1, beta2, sigma) %>%
  mutate(method = "Stan")

mh_compare_draws <- mh_draws %>%
  select(beta0, beta1, beta2, sigma) %>%
  mutate(method = "Custom MH")

compare_draws <- bind_rows(stan_m1_draws, mh_compare_draws) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value")

ggplot(compare_draws, aes(x = value, colour = method, fill = method)) +
  geom_density(alpha = 0.15) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior comparison: Stan vs Custom MH (Model 1)", x = NULL, y = "Densi
ty") +
  theme_minimal()
```



```
# Stan vs MH summary table
stan_table <- fit_m1$summary(variables = c("beta0", "beta1", "beta2", "sigma")) %>%
  as.data.frame() %>%
  select(variable, mean, median, sd, q5, q95) %>%
  mutate(method = "Stan")

mh_table2 <- mh_table %>%
  mutate(method = "Custom MH")

bind_rows(stan_table, mh_table2) %>%
  arrange(variable, method)
```

```
##   variable        mean      median          sd          q5         q95
## 1    beta0 13.73926741 13.73924289 0.006304899 13.72879678 13.74987281
## 2    beta0 13.73941273 13.73921300 0.006139360 13.72925830 13.74960830
## 3    beta1 -0.09273711 -0.09266675 0.006220925 -0.10324756 -0.08282485
## 4    beta1 -0.09291254 -0.09300522 0.006276566 -0.10294222 -0.08262237
## 5    beta2  0.06142306  0.06146047 0.006063590  0.05157917  0.07166787
## 6    beta2  0.06173576  0.06167671 0.006555387  0.05105858  0.07252791
## 7    sigma  0.53095776  0.53089151 0.004508547  0.52365559  0.53866101
## 8    sigma  0.53106841  0.53102125 0.004452695  0.52371207  0.53844117
##      method
## 1 Custom MH
## 2      Stan
## 3 Custom MH
## 4      Stan
## 5 Custom MH
## 6      Stan
## 7 Custom MH
## 8      Stan
```

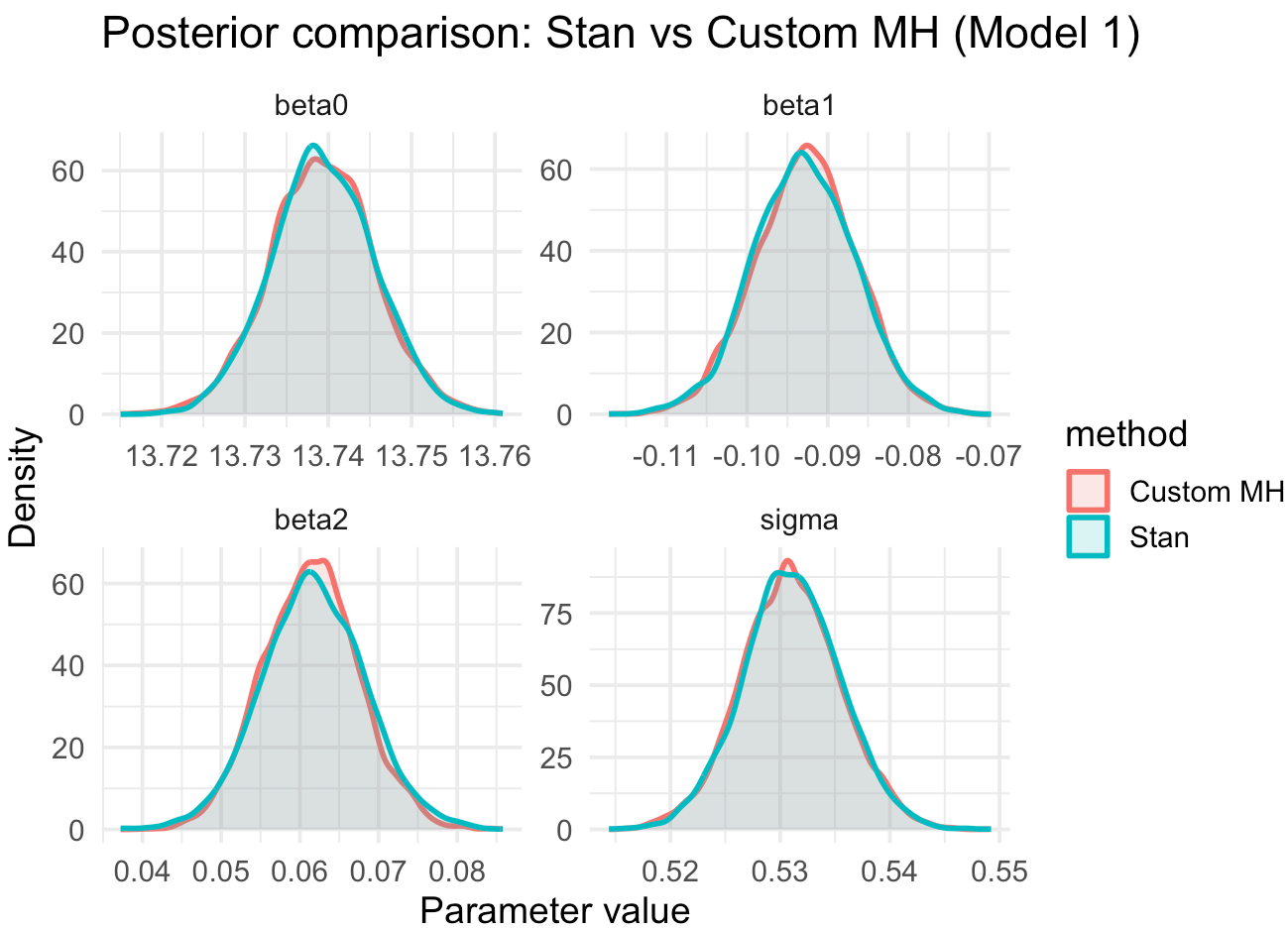
```
mh_all_draws <- bind_rows(
  lapply(seq_along(mh_draws_list), function(j) {
    mh_draws_list[[j]] %>%
      mutate(chain = j)
  })
)

stan_m1_summary <- fit_m1$summary(variables = c("beta0", "beta1", "beta2", "sigma")) %>%
  as.data.frame() %>%
  select(variable, mean, median, sd, q5, q95) %>%
  mutate(method = "Stan")

# Custom MH summary
mh_m1_summary <- mh_all_draws %>%
  summarise(
    beta0_mean = mean(beta0), beta1_mean = mean(beta1), beta2_mean = mean(beta2), sigma_
mean = mean(sigma),
    beta0_median = median(beta0), beta1_median = median(beta1), beta2_median = median(be
ta2), sigma_median = median(sigma),
    beta0_sd = sd(beta0), beta1_sd = sd(beta1), beta2_sd = sd(beta2), sigma_sd = sd(sigm
a),
    beta0_q5 = quantile(beta0, 0.05), beta1_q5 = quantile(beta1, 0.05), beta2_q5 = quant
ile(beta2, 0.05), sigma_q5 = quantile(sigma, 0.05),
    beta0_q95 = quantile(beta0, 0.95), beta1_q95 = quantile(beta1, 0.95), beta2_q95 = qu
antile(beta2, 0.95), sigma_q95 = quantile(sigma, 0.95)
  ) %>%
  pivot_longer(cols = everything(),
               names_to = c("variable", ".value"),
               names_sep = "_") %>%
  mutate(method = "Custom MH") %>%
  select(variable, mean, median, sd, q5, q95, method)

# Combined table
comparison_table <- bind_rows(mh_m1_summary, stan_m1_summary) %>%
  arrange(variable, method)

comparison_table
```

```
## # A tibble: 8 × 7
##   variable    mean  median      sd      q5     q95 method
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <chr>
## 1 beta0    13.7    13.7    0.00630 13.7    13.7    Custom MH
## 2 beta0    13.7    13.7    0.00614 13.7    13.7    Stan
## 3 beta1    -0.0927 -0.0927 0.00622 -0.103  -0.0828 Custom MH
## 4 beta1    -0.0929 -0.0930 0.00628 -0.103  -0.0826 Stan
## 5 beta2     0.0614  0.0615 0.00606  0.0516  0.0717 Custom MH
## 6 beta2     0.0617  0.0617 0.00656  0.0511  0.0725 Stan
## 7 sigma     0.531   0.531  0.00451  0.524   0.539  Custom MH
## 8 sigma     0.531   0.531  0.00445  0.524   0.538  Stan
```

```
plot_draws <- bind_rows(
  mh_all_draws %>% select(beta0, beta1, beta2, sigma) %>% mutate(method = "Custom MH"),
  fit_m1$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "df") %>%
    select(beta0, beta1, beta2, sigma) %>%
    mutate(method = "Stan")
) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma),
               names_to = "variable", values_to = "value")
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```

```
ggplot(plot_draws, aes(x = value, colour = method, fill = method)) +
  geom_density(alpha = 0.18, linewidth = 1) +
  facet_wrap(~ variable, scales = "free", ncol = 2) +
  labs(title = "Posterior comparison: Stan vs Custom MH (Model 1)",
       x = "Parameter value", y = "Density") +
  theme_minimal(base_size = 14)
```



Model 2

```
mh_step_model2 <- function(current, current_lp, proposal_sds, y, distance, buildingarea,
region, R, lp_fun) {
  proposal <- current + rnorm(length(current), mean = 0, sd = proposal_sds)
  proposal_lp <- lp_fun(proposal, y, distance, buildingarea, region, R)

  log_alpha <- proposal_lp - current_lp
  accepted <- log(runif(1)) < log_alpha

  if (accepted) {
    list(state = proposal, lp = proposal_lp, accepted = TRUE)
  } else {
    list(state = current, lp = current_lp, accepted = FALSE)
  }
}

mh_mixture_model2 <- function(y, distance, buildingarea, region, R,
                              init, lp_fun,
                              n_iter = 20000,
                              proposal_sds_small,
                              proposal_sds_large,
                              mix_prob_small = 0.5) {

  p <- length(init)
  samples <- matrix(NA_real_, nrow = n_iter, ncol = p)

  current <- init
  current_lp <- lp_fun(current, y, distance, buildingarea, region, R)

  accepted <- logical(n_iter)

  for (i in seq_len(n_iter)) {
    use_small <- runif(1) < mix_prob_small
    sds <- if (use_small) proposal_sds_small else proposal_sds_large

    out <- mh_step_model2(
      current = current,
      current_lp = current_lp,
      proposal_sds = sds,
      y = y,
      distance = distance,
      buildingarea = buildingarea,
      region = region,
      R = R,
      lp_fun = lp_fun
    )

    current <- out$state
    current_lp <- out$lp
    accepted[i] <- out$accepted
    samples[i, ] <- current
```

```
  }

  list(
    samples = samples,
    acceptance_rate = mean(accepted)
  )
}
```

```
set.seed(123)

R <- stan_data_model2$R
n_par <- 5 + R

init_fun_model2 <- function() {
  c(
    beta0 = 13.7,
    beta1 = -0.12,
    beta2 = 0.06,
    eta_sigma = log(0.50),
    eta_tau = log(0.25),
    rep(0, R)
  )
}

log_posterior_model2 <- function(theta, y, distance, buildingarea, region, R) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta_sigma <- theta[4]
  eta_tau <- theta[5]
  u_raw <- theta[6:(5 + R)]

  sigma <- exp(eta_sigma)
  tau <- exp(eta_tau)
  u <- tau * u_raw

  mu <- beta0 + u[region] + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, 0, 10, log = TRUE) +
    dnorm(beta1, 0, 1, log = TRUE) +
    dnorm(beta2, 0, 1, log = TRUE) +
    sum(dnorm(u_raw, 0, 1, log = TRUE)) +
    dexp(sigma, rate = 1, log = TRUE) + eta_sigma +
    dexp(tau, rate = 1, log = TRUE) + eta_tau

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}

proposal_sds_small_m2 <- c(
  0.025,
  0.0025,
  0.0025,
  0.015,
  0.035,
  rep(0.030, R)
)
```

```
proposal_sds_large_m2 <- c(
  0.060,
  0.0050,
  0.0050,
  0.030,
  0.070,
  rep(0.060, R)
)

mh_draws_model2_list <- vector("list", 4)
acc_model2 <- numeric(4)

for (ch in 1:4) {
  out <- mh_mixture_model2(
    y = stan_data_model2$y,
    distance = stan_data_model2$distance,
    buildingarea = stan_data_model2$buildingarea,
    region = stan_data_model2$region,
    R = stan_data_model2$R,
    init = init_fun_model2() + rnorm(n_par, 0, 0.08),
    lp_fun = log_posterior_model2,
    n_iter = 30000,
    proposal_sds_small = proposal_sds_small_m2,
    proposal_sds_large = proposal_sds_large_m2,
    mix_prob_small = 0.7
  )

  draws <- as.data.frame(out$samples)
  names(draws) <- c("beta0", "beta1", "beta2", "eta_sigma", "eta_tau", paste0("u_raw[",
1:R, "]"))
  draws$sigma <- exp(draws$eta_sigma)
  draws$tau <- exp(draws$eta_tau)
  draws$chain <- ch
  draws$iter <- 1:nrow(draws)

  mh_draws_model2_list[[ch]] <- draws
  acc_model2[ch] <- out$acceptance_rate
}

acc_model2
```

```
## [1] 0.07160000 0.05446667 0.07216667 0.06750000
```

```
mean(acc_model2)
```

```
## [1] 0.06643333
```

```
mh_all_draws_model2 <- bind_rows(mh_draws_model2_list)

dim(mh_all_draws_model2)
```

```
## [1] 120000     17
```

```
head(mh_all_draws_model2)
```

```
##      beta0      beta1     beta2  eta_sigma   eta_tau   u_raw[1]   u_raw[2]
## 1 13.66112 -0.1410364 0.1879336 -0.6751234 -1.377900 0.11367373  0.0148682
## 2 13.66112 -0.1410364 0.1879336 -0.6751234 -1.377900 0.11367373  0.0148682
## 3 13.68112 -0.1414463 0.1910409 -0.6891392 -1.364121 0.12578268 -0.0117249
## 4 13.64987 -0.1333302 0.1856905 -0.6385626 -1.381039 0.09769065 -0.0581036
## 5 13.64987 -0.1333302 0.1856905 -0.6385626 -1.381039 0.09769065 -0.0581036
## 6 13.64987 -0.1333302 0.1856905 -0.6385626 -1.381039 0.09769065 -0.0581036
##      u_raw[3]    u_raw[4]    u_raw[5]   u_raw[6]   u_raw[7]   u_raw[8]
## 1 -0.10768086 -0.06499561 -0.06822393 0.09536385 0.06090342 0.02769991
## 2 -0.10768086 -0.06499561 -0.06822393 0.09536385 0.06090342 0.02769991
## 3 -0.14724899 -0.06413029 -0.08118783 0.14606002 0.09775521 0.03598061
## 4 -0.01825383 -0.14419151 -0.05143560 0.22009860 0.13581693 0.06070195
## 5 -0.01825383 -0.14419151 -0.05143560 0.22009860 0.13581693 0.06070195
## 6 -0.01825383 -0.14419151 -0.05143560 0.22009860 0.13581693 0.06070195
##       sigma       tau chain iter
## 1 0.5090936 0.2521073     1    1
## 2 0.5090936 0.2521073     1    2
## 3 0.5020080 0.2556054     1    3
## 4 0.5280509 0.2513173     1    4
## 5 0.5280509 0.2513173     1    5
## 6 0.5280509 0.2513173     1    6
```

```
burn_in_m2 <- 15000
mh_model2_post_list <- lapply(mh_draws_model2_list, function(df) {
  df %>% filter(iter > burn_in_m2)
})

mh_post_model2 <- bind_rows(mh_model2_post_list)
```

```
# Posterior summaries for main parameters
mh_summary_model2 <- mh_post_model2 %>%
  summarise(
    beta0_mean = mean(beta0), beta0_q5 = quantile(beta0, 0.05), beta0_q95 = quantile(bet
a0, 0.95),
    beta1_mean = mean(beta1), beta1_q5 = quantile(beta1, 0.05), beta1_q95 = quantile(bet
a1, 0.95),
    beta2_mean = mean(beta2), beta2_q5 = quantile(beta2, 0.05), beta2_q95 = quantile(bet
a2, 0.95),
    tau_mean   = mean(tau),   tau_q5   = quantile(tau,   0.05), tau_q95   = quantile(ta
u,   0.95),
    sigma_mean = mean(sigma), sigma_q5 = quantile(sigma, 0.05), sigma_q95 = quantile(sig
ma, 0.95)
  )

mh_summary_model2
```

```
##   beta0_mean beta0_q5 beta0_q95 beta1_mean   beta1_q5  beta1_q95 beta2_mean
## 1   13.67581 13.50497   13.8958 -0.1180948 -0.1314767 -0.1042796 0.06166954
##    beta2_q5  beta2_q95  tau_mean    tau_q5  tau_q95 sigma_mean  sigma_q5
## 1 0.0517039 0.07132264 0.3172151 0.1981605 0.555753  0.4948531 0.4882091
##   sigma_q95
## 1 0.5016155
```

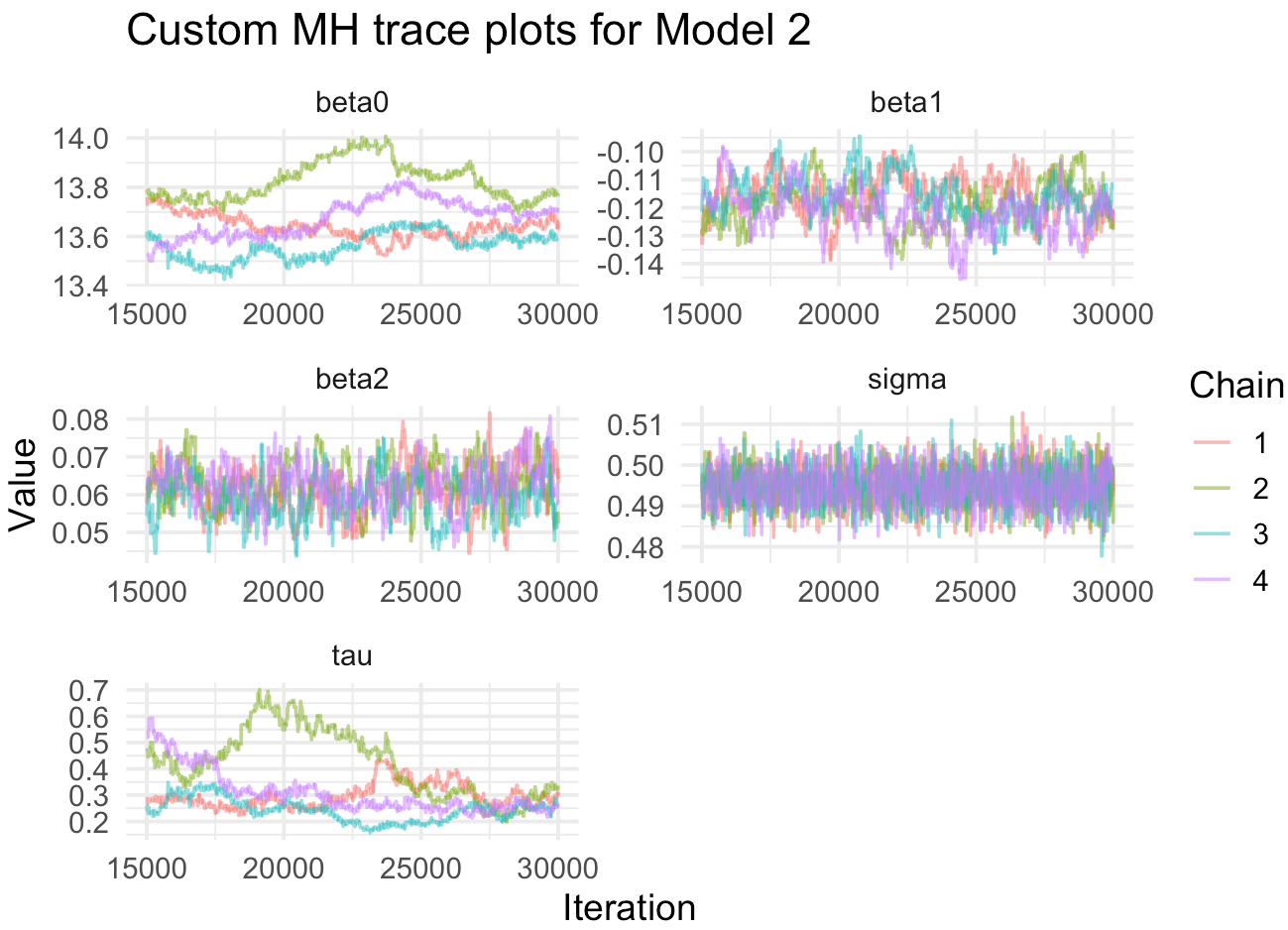
```
# chain-specific data
mh_chain_list_model2 <- lapply(mh_draws_model2_list, function(df) {
  df %>%
    filter(iter > burn_in_m2) %>%
    select(beta0, beta1, beta2, tau, sigma)
})

mh_mcmc_list_model2 <- coda::mcmc.list(
  lapply(mh_chain_list_model2, coda::mcmc)
)

# convergence diagnostics
gelman.diag(mh_mcmc_list_model2, autoburnin = FALSE)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## beta0       2.51       4.31
## beta1       1.10       1.28
## beta2       1.07       1.21
## tau         1.61       2.91
## sigma       1.00       1.01
##
## Multivariate psrf
##
## 2.72
```

```
effectiveSize(mh_mcmc_list_model2)
```

```
##      beta0      beta1      beta2        tau      sigma
##   15.84692  117.96572  206.11546   20.90179 1649.37495
```

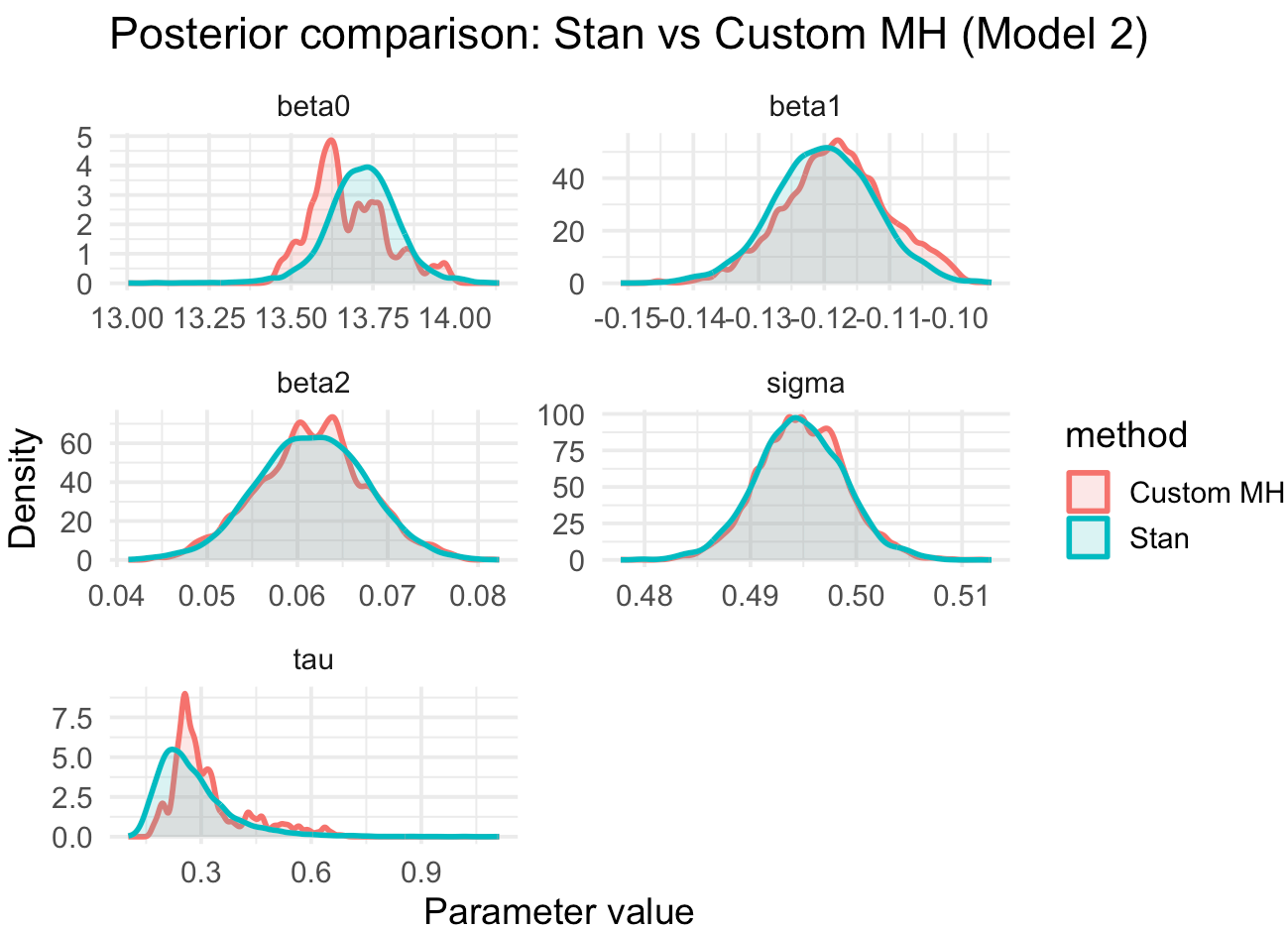
```
mh_trace_model2 <- mh_post_model2 %>%
  select(iter, chain, beta0, beta1, beta2, tau, sigma) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, tau, sigma),
               names_to = "variable", values_to = "value")

ggplot(mh_trace_model2, aes(x = iter, y = value, colour = factor(chain))) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ variable, scales = "free", ncol = 2) +
  labs(
    title = "Custom MH trace plots for Model 2",
    x = "Iteration",
    y = "Value",
    colour = "Chain"
  ) +
  theme_minimal(base_size = 14)
```




```
stan_draws_model2 <- fit_m2$draws(
  variables = c("beta0", "beta1", "beta2", "tau", "sigma"),
  format = "df"
) %>%
  select(beta0, beta1, beta2, tau, sigma) %>%
  mutate(method = "Stan")
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```

```
mh_draws_compare_model2 <- mh_post_model2 %>%
  select(beta0, beta1, beta2, tau, sigma) %>%
  mutate(method = "Custom MH")

comparison_draws_model2 <- bind_rows(
  mh_draws_compare_model2,
  stan_draws_model2
) %>%
  pivot_longer(
    cols = c(beta0, beta1, beta2, tau, sigma),
    names_to = "variable",
    values_to = "value"
  )

ggplot(comparison_draws_model2, aes(x = value, colour = method, fill = method)) +
  geom_density(alpha = 0.18, linewidth = 1) +
  facet_wrap(~ variable, scales = "free", ncol = 2) +
  labs(
    title = "Posterior comparison: Stan vs Custom MH (Model 2)",
    x = "Parameter value",
    y = "Density"
  ) +
  theme_minimal(base_size = 14)
```



```
mh_summary_table_model2 <- mh_post_model2 %>%
  summarise(
    beta0_mean = mean(beta0), beta1_mean = mean(beta1), beta2_mean = mean(beta2),
    tau_mean = mean(tau), sigma_mean = mean(sigma),
    beta0_q5 = quantile(beta0, 0.05), beta1_q5 = quantile(beta1, 0.05), beta2_q5 = quant
ile(beta2, 0.05),
    tau_q5 = quantile(tau, 0.05), sigma_q5 = quantile(sigma, 0.05),
    beta0_q95 = quantile(beta0, 0.95), beta1_q95 = quantile(beta1, 0.95), beta2_q95 = qu
antile(beta2, 0.95),
    tau_q95 = quantile(tau, 0.95), sigma_q95 = quantile(sigma, 0.95)
  ) %>%
  pivot_longer(everything(),
               names_to = c("variable", ".value"),
               names_pattern = "(beta0|beta1|beta2|tau|sigma)_(mean|q5|q95)") %>%
  mutate(method = "Custom MH")

stan_summary_table_model2 <- fit_m2$summary(
  variables = c("beta0", "beta1", "beta2", "tau", "sigma")
) %>%
  as.data.frame() %>%
  select(variable, mean, q5, q95) %>%
  mutate(method = "Stan")

comparison_table_model2 <- bind_rows(
  mh_summary_table_model2,
  stan_summary_table_model2
)

comparison_table_model2
```

```
## # A tibble: 10 × 5
##    variable    mean      q5     q95 method
##    <chr>      <dbl>   <dbl>   <dbl> <chr>
##  1 beta0    13.7    13.5    13.9    Custom MH
##  2 beta1    -0.118  -0.131  -0.104  Custom MH
##  3 beta2     0.0617  0.0517  0.0713 Custom MH
##  4 tau       0.317   0.198   0.556  Custom MH
##  5 sigma     0.495   0.488   0.502  Custom MH
##  6 beta0    13.7    13.5    13.9    Stan
##  7 beta1    -0.120  -0.132  -0.108  Stan
##  8 beta2     0.0616  0.0519  0.0713 Stan
##  9 tau       0.280   0.164   0.476  Stan
## 10 sigma     0.495   0.488   0.501  Stan
```